

一. 编译概述

编译是指将高层次语言代码转换成低层次语言代码的过程

AI编译 高层次AI模型 低层次执行表示

源程序 $\rightarrow$ 扫描/词法分析 $\rightarrow$ tokens $\rightarrow$ 解析/语法分析 $\rightarrow$ 语法树/AST $\rightarrow$ 语义分析 $\rightarrow$ 中间语言IR $\rightarrow$ 优化 $\rightarrow$ 汇编/机器码/字节码 $\leftarrow$ A码生成

Lexer Parser Semantic Analysis

编译技术应用: 编译器、解释器、虚拟机、IOE、静态分析、安全分析、性能优化...

前端(扫描、解析): 理解程序 $\rightarrow$ 中端(语义): IR+分析+优化, 尽量机器无关 $\rightarrow$ 后端(目标语言代码生成): 低层
IR的意义: 解耦前/后端; 多语言共享优化器; 多目标平台共享后端框架; 便于控制流等分析。

语言例子:

C/Rust: 常AOT到机器码 Java: Source $\rightarrow$ bytecode $\rightarrow$ JVM JavaScript/PHP: 动态类型, 解释/JIT常见
TypeScript: 静态类型检查 $\rightarrow$ JavaScript

Rust ownership: value = owner + scope + value

语义分析: 判断“值能否被使用”等
(同一时刻, value 仅有一个 owner, &mut T 只能一个且不与 &T 共存
owner 离开 scope, value 被释放)

语言分类: 按执行方式 { 命令式, 函数式, 声明式 } 按运行过程 { 编译性: 机器码, 翻译性: 指令 } 按数据类型 { 静态类型: 编译期才知道类型, 动态类型: 运行期才确定类型 }

AOT: 运行前编译, 启动/执行直接, 但缺少运行时信息
JIT: 运行时编译热点代码, 可利用运行时信息优化, 但有编译开销

$\langle FN \rangle \langle IDENT, 名字 \rangle \langle L/RPAR \rangle$
 $\langle L/RBRACE \rangle \langle LET \rangle \langle EQ \rangle \langle NUM \rangle$
 $\{ \} \langle SEMI \rangle ;$

二. 扫描和词法分析

扫描(词法分析): 输入字符串形式的源代码 输出划词分割之后的 token 流(内容+类别)
任务: 识别关键字、标识符、数字、运算符... 跳过空白、注释...

Token = 类型 + 文本 + 属性 (例: $\langle IDENT, x \rangle$)

规则可用正则表达式描述, 可转换为 NFA $\rightarrow$ DFA, 高效实现扫描器

正则表达式形式: $r|s$ 选择 rs 连接 r^* 0次或多次 r^+ 1次或多次 $r!$ 0个或1个 $[0.9]$ 字符集合

$[^a-z]$ 非 a-z . 任意 $a\{m,n\}$ m到n次

优先级: $* + ? \{m,n\} > 连接 > |$

性质: 1. 满足交换、结合 连接满足结合 连接对|分配 $(r^+)^* = r^+$ 连接不交换

词法分析: 正则语言 有限自动机 FA 语法分析: 上下文无关语言 CFG, 下推自动机 PDA

DFA: 每个 state + input 至多一个后继 易执行 } 表达能力等价
NFA: 可有多后继、epsilon 边 易构造
DFA: (状态集 S, 输入表 Σ , 转移函数 δ , 初态 S_0 , 接受状态集合 F)

正则表达式转 NFA:

$S \rightarrow (P \xrightarrow{\epsilon} (FNS) \xrightarrow{\epsilon} FNT) \rightarrow Q$ $S \rightarrow (P \xrightarrow{\epsilon} FNS) \xrightarrow{\epsilon} FNT \rightarrow Q$ $S \rightarrow (P \xrightarrow{\epsilon} FA(S) \xrightarrow{\epsilon} F) \rightarrow Q$

NFA $\rightarrow$ DFA 子集构造: DFA 状态 = NFA 状态集合 $D_0 = E-closure(\{S_0\})$ $U = E-closure(move(T, a))$ U 新加入
 $E-closure(T)$: 从 T 出发只走 E 边可达的所有状态, 包括 T 本身 (取一个未处理 T U 含 NFA 结, 也是 DFA
 $move(T, a)$: 从 T 出发读 a 一步可达 (对所有 a)

多 token 合并: 新 start $\xrightarrow{\epsilon}$ 每个 token NFA 的 start, 合并 NFA $\rightarrow$ DFA

token 冲突处理: 最长匹配、同长度按规则优先级

字符串/注释: 状态机处理, 遇结束符退出 空白和注释: 一般跳过, 不生成 token

三. 解析和语法分析

解析 (Parse): 以 token 序列为输入, 应用语法规则来生成语法解析树 (关心 token 是否按规则排列)

区分: 词法分析: 字符组成 token 是否合法

描述 token 如何组成表达式、语句、函数、程序等结构

语法分析: token 排列是否合规

(例) $Assign \rightarrow left = Expr$ $Expr \rightarrow Expr * Term \mid Term$

BNF: $A ::= \alpha \mid \beta$

终结符: token

非终结符: V

产生式 R , 开始符号 S

非终结符
(语法结构)

推导: 从 S 出发, 反复应用产生式, 生成终结符号串

(最左/右推导: 每次替换最左/右非终结符)

Parse Tree: 根为开始符号 内部节点为非终结符 叶子为终结符 (表示 token 串的语法推导结构)

完整反映文法推导, 结构详细 而 AST: 只保留语义结构

二义性: 同一 token 串存在两种推导 解决: 把优先级写进文法 (把优先的放更深)

表达式文法

结合性: 同优先级从左还是从右结合

左递归: $A \rightarrow A \alpha \mid \beta$ (可推出以自己开头的形式) 会无限递归

$E \rightarrow E + T \mid T$

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow T * F \mid F$

$\Rightarrow$

$T \rightarrow FT'$

$F \rightarrow (E) \mid id \mid num$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id \mid num$

直接消除: $A \rightarrow A \alpha \mid \beta$ 改为 $A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \epsilon$

左提取: $A \rightarrow \alpha \beta_1 \mid \alpha \beta_2$ 改为 $A \rightarrow \alpha A'$ $A' \rightarrow \beta_1 \mid \beta_2$

预测分析目标: 根据当前 token 输入选择唯一产生式 (为此需: 消除左递归、左提取、二义性)

FIRST(α): α 能推出的串的首个终结符号集合 (可为空则含 ϵ)

FOLLOW(A): 可紧跟在非终结符 A 后的终结符号集合

山(1)直见: 同一非终结符的不同候选的 FIRST 集不相交, 有 ϵ 则还依赖开始符号 $S: \epsilon \in FOLLOW(S)$

常见语法规则: $VarDecl \rightarrow Type\ id \mid Type\ id = Expr$ $FuncDecl \rightarrow Type\ id (Params)\ Block$

$If \rightarrow if (Expr)\ Stmt\ ElsePart$ $ElsePart \rightarrow else\ stmt \mid \epsilon$ $While \rightarrow while (Expr)\ Stmt$

D. 抽象语法树及其运用

AST: 由 Parse Tree 简化而来; 去掉括号、分号、中间非终结符等冗余语法细节; 保留表达式、语句、声明等语义结构

常见 AST 节点: Program、FunctionDecl、Block、Assign、If/While/For/Return、VarDecl、CallExpr、BinaryExpr

AST 构造: 运算符作为内部节点, 操作数为子节点; 括号只影响结构; 分/逗号通常不保留 Identifier、Literal

AST 用途: 作用域分析、符号表、变量绑定、类型检查、类型推断、生成 IR、优化

作用域 scope: 决定标识符在哪些程序区域内可见/有效

符号表 symbol table: 记录标识符信息, 如 name、kind、type、scope、params、location

作用域栈: 新作用域则压入新符号表 有声明则插入当前作用域 (ID 编号区分同名) 遇使用: 从当前域向外找

例:

```
int x = 0;
void f() {
  int y = x;
  f();
  int x = 2;
  y = x;
  x = y;
}
```



```
global
├ x: int (id=0)
├ f
├   y: int (id=1)
├   └ y: int 2
├   └ y: int 3 (1处的x值)
├   └ x: int 4
├   └   y 5 (4处的x值, 3处y)
├   └   x 6 (1处x, 5处y值)
└ x
```

类型推断: 根据上下文, 自动推导自变量/表达式类型 思路: 生成类型约束 $\rightarrow$ 求解约束

弱类型语言: 步隐式转换

强类型语言: 一般不允隐式转换

AST + 符号表 + 类型信息 $\rightarrow$ IR 生成

(语义检查失败时, 不应正常生成目标代码)

五、中间语言 IR

AST 接近 源代码, 适合作 语义分析

IR 更底层、更统一, 适合 优化和代码生成

IR: 由指令和地址做成, 变量名、常量、编译器生成的临时变量或存储单元

规范: 三地址码 $x = y \text{ op } z$ 例 $a = b + c * d \rightarrow t_1 = c * d \quad t_2 = b + t_1 \quad a = t_2$

LLVM: 编译器基础设施 前端 $\rightarrow$ LLVM IR $\rightarrow$ 优化器 $\rightarrow$ 后端

$\hookrightarrow$ 强类型、低层、仍可读、适合 优化和 多后端代码生成

LLVM IR 基本形式

$\%result = \text{op type operand1, operand2}$

define return-type @func(param-types $\% \sim, \dots$) {

label: entry

语句...

ret type value

}

op: $-$ sub, $+$ add, $*$ mul, $/$ sdiv, $/$ udiv

有符号除

br { 无条件 } br label %L

{ 条件 } br if %cond, label %T, label %F

if 的 IR 模板:

$\%cond = \text{icmp}$

br if $\%cond$, label $\%then$, label $\%else$

then:

...

br join

else:

...

br join

join:

...

常见类型: i1 (布尔/条件) i32 i64 float double

区分变量 < 地址
值

load type, ptr ~ 取出值

store type 值, ptr ~ 写入内存

allocate: 分配局部变量空间

eq, ne, slt, sle, sgt, sge

$\nearrow = < \leq > \geq$

icmp 比较指令, 结果 i1 icmp 谓词 type $\%a, \%b$

call: $\%r = \text{call ret-type @func(arg-type arg, ...)$

(void)

逻辑运算: 无需专门定义“与”“非”

$\%T = \text{xor if } \%b, \text{true} \rightarrow !\%b$

while 的 IR 模板:

br label $\%cond$

cond:

$\%C = \text{icmp}$

br if $\%C$, label $\%body$, label $\%exit$

body:

...

exit:

...

类型转换: zext 零扩展 sext 符号扩展 trunc 截断 sitofp int $\rightarrow$ float fptosi float $\rightarrow$ int

IR 通常组织成基本块: 单入口/出口; 中间顺序执行; 末尾通常是 br/ret。CFG 以基本块为节点, 跳转为边。

GIR: 面向语义逻辑的中间语言, 简单、忠实表达语言 目标是统一表示多语言, 支持分析、优化和代码生成

数据语句: $\text{target} = \text{operand1 op operand2}$ $\text{target} = \text{array}[\text{index}]$ $\text{array}[\text{index}] = \text{source}$

$\text{target} = \text{object}.\text{field}$ $\text{object}.\text{field} = \text{source}$

控制流: if(condition) true-body else falsebody while(condition) loop-body

for(init; condition; update) body for element in receiver body

(底层: 条件、基本块、跳转)

AST 转 IR: 自顶向下翻译

表达式: 递归生成左右子表达式, 再生成临时变量

赋值: 先生成右值, 再写入左值

if、while 根据模板来

六、语义分析和应用

控制流分析：程序语句可能按什么顺序执行

数据流分析：定义、使用、值等信息如何沿控制流传播

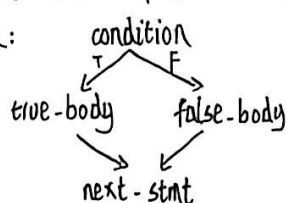
Control flow：描述程序语句/基本块之间可能的执行顺序

顺序、分支、循环、跳转都会形成控制流边

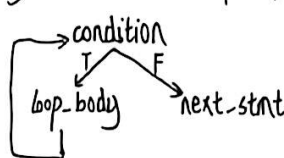
CFG：控制流图 节点是基本块 边是可能的执行转移

数据流分析通常在CFG上进行

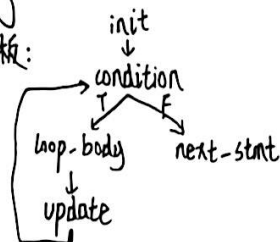
if模板：



while模板：



for模板：



Data flow：在CFG上传播程序事实，计算每个节点入口/出口处的信息 (找出变量间的依赖关系)

def：一条指令定义了什么

(赋值一般def左，use右)

use：一条指令用到了什么

In：基本块入口处的数据流事实 (进入之前可用)

Out：基本块出口处的数据流事实 (经过后仍可用)

数据流迭代：初始化In/Out 若有集合变化：对所有块计算In/Out，更新 直到不再变化

到达定义：定义d：x=... 能到达某点，当且仅当存在从d到该点的路径，且路上x未被重新定义

前向may分析 (存在一条路即可)：

$$In[B] = \bigcup Out[p], p \text{ 为前驱} \quad Out[B] = Gen[B] \cup (In[B] - Kill[B])$$

↳ 所有前驱Out之并

↓ B内新定义

↓ B内重新定义，杀死了旧定义

活跃变量：变量v在某点live，则从该点出发存在路径会使用v，且使用前v未被重新定义

后向may分析 (看“未来会不会用”)：

$$Out[B] = \bigcup In[s], s \text{ 为后继} \quad In[B] = Use[B] \cup (Out[B] - Def[B])$$

↳ 所有后继In之并

↓ B内使用

↓ B内自己定义

forward：信息沿CF传播

backward：信息逆CF传播

may：存在即可，∪会并

must：所有都成立，∩会并

代码优化应用：

常量传播：若某点能确定变量为常量，则替换

死代码删除：若某定义产生值之后不再使用，无副作用，可删

不可达代码：CFG中从入口处无法到达的代码 (删或警告)

(活跃变量可辅助判断)

简单补充：

函数间分析：call-graph 跨函数分析

指针分析：计算指针可能指向的对象集合

处理别名

判断指针读写影响

SSA：每个变量只定义一次，分支汇合用phi，让def-use清晰

二、后端

IR → 汇编/机器码

ISA：CPU的指令集架构

RISC：简单，load/store风格，ARM/RISC-V
CISC：复杂，x86